Lab 41 — Incident Response: Live Triage

> **SY0-701 Objective 4.8** — Explain appropriate incident response activities.
>
> **Domain 4** — Security Operations (28%) · **Time:** ~15 min · **Course:** TGS-2023039181 CompTIA Security+ (SY0-701)

Goal: take a simulated host incident through every phase of the incident response process, collecting evidence in order of volatility before you contain it.

You'll build: a case folder with hashed volatile evidence, a contained and eradicated incident (an unexpected listener, a rogue account and a cron persistence job), a recovery check and a lessons-learned report with a root cause analysis.

Prerequisites: Killercoda Ubuntu Playground (root shell) — https://killercoda.com/playgrounds/scenario/ubuntu ; Lab 37 (EDR/osquery) and Lab 38 (accounts) are useful background; tools installed in step 1.

Background

- The SY0-701 process is **preparation, detection, analysis, containment, eradication, recovery, lessons learned**. NIST SP 800-61 groups the same work into fewer phases; the exam uses the seven-step list.
- **Preparation** happens before any incident: the IR plan, a trained team, a communication plan, a jump bag of tools and playbooks. **Training** and **testing** (tabletop exercises, walkthroughs, simulations) keep the plan current.
- Collect volatile evidence **before** containment changes it. Killing a process or pulling the cable destroys memory and connection state that the analysis needs.
- **Containment** limits the damage (isolate, block, disable). **Eradication** removes the cause (malware, persistence, rogue accounts). **Recovery** restores the service and monitors it closely.
- **Lessons learned** includes a **root cause analysis**, for example the five whys, and feeds improvements back into preparation. **Threat hunting** then looks proactively for the same indicators on other hosts.

> Safety: every "indicator" in this lab is a harmless stand-in. The listener serves an empty folder on 127.0.0.1 only, and the cron job only appends a line to a local file.

Steps

1. Prepare: install the jump-bag tools and open a case

Preparation means the tools, the case folder and the record-keeping exist before you need them.

```
apt-get update -qq && DEBIAN_FRONTEND=noninteractive apt-get install -y cron procps
iproute2 psmisc lsof python3
```

```
systemctl enable --now cron
mkdir -p /evidence/IR-2026-041
cat > /evidence/IR-2026-041/case.txt <<EOF
Case ID:   IR-2026-041
Opened:    $(date -u +%FT%TZ)
Handler:   $(whoami)@$(hostname)
Severity:  TBD (set after analysis)
Playbook:  Suspected host compromise v1.2
EOF
cat /evidence/IR-2026-041/case.txt
```

2. Simulate the incident (exercise set-up)

The exercise facilitator plants three harmless indicators: a rogue local account, an unexpected listening service and a cron job for persistence. In a real incident you would discover these, not create them.

```
id svc-update >/dev/null 2>&1 || useradd -m -s /bin/bash -c "updater" svc-update
mkdir -p /tmp/.cache-www
cat > /tmp/.update.sh <<'EOF'
#!/bin/sh
echo "marker $(date -u +%FT%TZ)" >> /tmp/.update.marker
EOF
chmod +x /tmp/.update.sh
echo '* * * * * /tmp/.update.sh' | crontab -u svc-update -
nohup su -s /bin/sh svc-update -c 'python3 -m http.server 4444 --bind 127.0.0.1 --
directory /tmp/.cache-www' >/dev/null 2>&1 &
sleep 2; echo "exercise indicators planted"
```

3. Detect: confirm the alert and list the indicators

An alert or a user report starts detection. Confirm it by checking listening ports, processes, recently created accounts and scheduled tasks. Write each indicator of compromise (IoC) into the case notes as you find it.

```
ss -tlnp | grep 4444
ps -eo pid,ppid,user,lstart,cmd | grep '[h]ttp.server 4444'
awk -F: '$3 >= 1000 && $3 < 65000 {print $1, $3, $6, $7}' /etc/passwd
for u in $(awk -F: '$3 >= 1000 && $3 < 65000 {print $1}' /etc/passwd); do crontab -u "$u"
-l 2>/dev/null | sed "s/^/$u: /"; done
ls -la /tmp | grep ' \.'
```

```
LISTEN 0 5 127.0.0.1:4444 0.0.0.0:* users:(("python3",pid=4242,fd=3))
svc-update 1001 /home/svc-update /bin/bash
svc-update: * * * * * /tmp/.update.sh
```

4. Analyse: collect volatile evidence in order of volatility, then hash it

Capture the most volatile state first (processes, network connections, logged-in users), then less volatile items (logs, cron tables, the suspicious script). Hash everything straight away so you can later prove nothing changed.

```
cd /evidence/IR-2026-041
date -u +%FT%TZ > 00-collection-start.txt
ps auxwwf > 01-processes.txt
```

```
ss -tunap > 02-sockets.txt
ip neigh > 03-arp.txt; ip route > 03-routes.txt
who -a > 04-who.txt; last -n 20 > 04-last.txt 2>/dev/null || true
lsof -nP -i :4444 > 05-lsof-4444.txt 2>/dev/null || true
crontab -u svc-update -l > 06-cron-svc-update.txt
cp /tmp/.update.sh 07-update.sh.evidence
journalctl --since "-30 min" --no-pager > 08-journal-30min.txt
sha256sum 0* > evidence.sha256 && cat evidence.sha256
```

5. Contain: stop the spread without destroying evidence

Short-term containment disables the account, kills the account's sessions and blocks the port. The evidence you hashed in step 4 is already safe.

```
usermod -L -e 1 svc-update
iptables -I INPUT -p tcp --dport 4444 -j DROP
pkill -u svc-update || true
sleep 1; ss -tlnp | grep 4444 || echo "listener stopped"
passwd -S svc-update
```

6. Eradicate: remove the cause and the persistence

Remove the persistence mechanism, the dropped script and the rogue account. Keep the evidence copies in the case folder.

```
crontab -u svc-update -r 2>/dev/null || true
rm -f /tmp/.update.sh /tmp/.update.marker
rm -rf /tmp/.cache-www
userdel -r svc-update 2>/dev/null || userdel svc-update
iptables -D INPUT -p tcp --dport 4444 -j DROP
echo "eradication complete"
```

7. Recover: validate that the host is clean and keep watching

Recovery puts the system back into service only after you verify it, then monitors it more closely for a period in case the attacker returns.

```
ss -tlnp | grep -q 4444 && echo "FAIL: listener still present" || echo "OK: no listener on
4444"
id svc-update 2>/dev/null && echo "FAIL: account still present" || echo "OK: rogue account
removed"
ls /var/spool/cron/crontabs/ 2>/dev/null | grep -q svc-update && echo "FAIL: cron remains"
|| echo "OK: no cron persistence"
ls /tmp/.update.sh 2>/dev/null || echo "OK: dropped script removed"
cd /evidence/IR-2026-041 && sha256sum -c evidence.sha256
```

8. Lessons learned: root cause analysis and the incident report

Hold the review within a few days, while memories are fresh. Use the five whys to reach the root cause, record what went well and what did not, and feed actions back into preparation. Hash the report as well.

```
cat > /evidence/IR-2026-041/report.md <<EOF
# Incident IR-2026-041 - post-incident report
```

```
- Detected: $(head -1 /evidence/IR-2026-041/00-collection-start.txt) (unexpected listener
alert)
- IoCs: TCP 127.0.0.1:4444 listener; rogue account svc-update; cron job /tmp/.update.sh
every minute
- Containment: account locked and expired, sessions killed, port 4444 blocked
- Eradication: cron entry, script and account removed
- Recovery: no listener, account or persistence remains; evidence hashes verified
## Root cause (five whys)
1. Why was there a rogue account? It was created by someone with root access.
2. Why did they have root access? A shared admin password was reused on several hosts.
3. Why was it shared? No privileged access management or JIT elevation was in place.
4. Why was creation not noticed? No alert on new UIDs or new crontabs.
5. Why no alert? Account-creation monitoring was never added to the SIEM use cases.
## Actions
- Deploy PAM with password vaulting and JIT elevation (Lab 38).
- Alert on new local accounts, new listening ports and new crontabs (Labs 33 and 37).
- Run a tabletop exercise of this playbook with the service owners next quarter.
EOF
sha256sum /evidence/IR-2026-041/report.md | tee -a /evidence/IR-2026-041/evidence.sha256
```

Test it

```
ls /evidence/IR-2026-041/
sha256sum -c /evidence/IR-2026-041/evidence.sha256 | grep -c ': OK'
ss -tlnp | grep -c 4444
```

Expected: the case folder holds the numbered evidence files, `report.md` and `evidence.sha256`; every hash line verifies `OK`; and the listener count is `0`.

Troubleshooting

Symptom	Fix
`crontab: command not found` or the cron job never runs	Install and start cron: `apt-get install -y cron && systemctl enable --now cron`.
`userdel: user svc-update is currently used by process`	A process is still running as that user. Run `pkill -KILL -u svc-update`, then repeat step 6.
`sha256sum -c` reports FAILED	An evidence file was changed after hashing. This is exactly what hashing detects; record it in the case notes and re-collect from the source if possible.

Challenge

Turn this lab into a tabletop exercise: write four "injects" (for example "the listener reappears on a second host" or "a journalist calls the press office") and, for each, decide which phase you are in, who is notified and what the playbook says to do next.

Reflection

Why did you collect processes and sockets in step 4 before killing anything in step 5, and which item of evidence would you have lost if you had contained first?

Exam tips

- Order: preparation, detection, analysis, containment, eradication, recovery, lessons learned. "Isolate the host" is containment; "remove the malware" is eradication; "restore from backup" is recovery.
- Tabletop = discussion only, no systems touched; simulation = a hands-on exercise with realistic, synthetic events.
- Root cause analysis happens in lessons learned; threat hunting is proactive and assumes a compromise may already exist.
- Collect volatile data first (CPU, memory, network state) before shutting down or isolating a host.

References

- NIST SP 800-61 Incident Response Recommendations — https://csrc.nist.gov/pubs/sp/800/61/r3/final
- SANS incident handler's handbook — https://www.sans.org/white-papers/33901/
- MITRE ATT&CK Persistence (TA0003) — https://attack.mitre.org/tactics/TA0003/
- CISA incident response playbooks — https://www.cisa.gov/resources-tools/resources/federal-government-cybersecurity-incident-and-vulnerability-response-playbooks

---